Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan VII

```
# To create tree
tree = {}
path = []
maxHeight, maxHeightNode = -1, -1

# Function to store the path from given vertex to the target vertex in a vector path
```

```
def getDiameterPath(vertex, targetVertex, parent, path):

    # If the target node is found, push it into path vector
    if (vertex == targetVertex):
        path.append(vertex)
        return True

    for i in range(len(tree[vertex])):
        # To prevent visiting a node already visited
        if (tree[vertex][i] == parent):
            continue

        # Recursive call to the neighbours of current node inorder to get the path
        if (getDiameterPath(tree[vertex][i], targetVertex, vertex, path)):
            path.append(vertex)
            return True
    return False
```

```
# Function to obtain and return the farthest node from a given vertex
def farthestNode(vertex, parent, height):
    global maxHeight, maxHeightNode

    # If the current height is maximum so far, then save the current node
    if (height > maxHeight):
        maxHeight = height
        maxHeightNode = vertex

    # Iterate over all the neighbours of current node
    if (vertex in tree):
        for i in range(len(tree[vertex])):

            # This is to prevent visiting a already visited node
            if (tree[vertex][i] == parent):
                continue

            # Next call will be at 1 height higher than our current height
            farthestNode(tree[vertex][i], vertex, height + 1)
```

```
# Function to add edges
def addedge(a, b):
    if (a not in tree):
        tree[a] = []

    tree[a].append(b)

    if (b not in tree):
        tree[b] = []

    tree[b].append(a)
```

```
def FindCenter(n):
    # Now we will find the 1st farthest node from 0(any arbitrary node)

    # Perform DFS from 0 and update the maxHeightNode to obtain the farthest node from 0

    # Reset to -1
    maxHeight = -1

    # Reset to -1
    maxHeightNode = -1

    farthestNode(0, -1, 0)
```

```
        # Stores one end of the diameter
        leaf1 = maxHeightNode

        # Similarly the other end of the diameter
        # Reset the maxHeight
        maxHeight = -1
        farthestNode(maxHeightNode, -1, 0)

        # Stores the second end of the diameter
        leaf2 = maxHeightNode

        # Store the diameter into the vector path
        path = []

        # Diameter is equal to the path between the two farthest nodes leaf1 and leaf2
        getDiameterPath(leaf1, leaf2, -1, path)

        pathSize = len(path)

        if (pathSize % 2 == 1):
            print(path[int(pathSize / 2)]*-1)
        else:
            print(path[int(pathSize / 2)], ", ", path[int((pathSize - 1) / 2)], sep = "", end = "")
```

```
N = 4

tree = {}
addedge(1, 0)
addedge(1, 2)
addedge(1, 3)

FindCenter(N)

# This code is contributed by suresh07.
```

```
1
```

Saya akan analisis kode ini dulu secara menyeluruh sebelum menjelaskan logikanya.

Analisis Logika Program

Program ini mencari **center (pusat) dari sebuah pohon (tree)** menggunakan algoritma berbasis diameter pohon.

Alur kerja:

1. Bangun tree sebagai adjacency list
2. Dari node sembarang (0), cari node terjauh → `leaf1`
3. Dari `leaf1`, cari node terjauh → `leaf2`
4. `leaf1` ke `leaf2` adalah **diameter** pohon
5. Titik tengah diameter = **center** pohon

Bug yang Ditemukan

Bug 1 — `FindCenter` mengeset `maxHeight` dan `maxHeightNode` sebagai variabel lokal, tapi `farthestNode` membaca variabel **global** dengan nama yang sama. Akibatnya reset tidak berpengaruh ke global.

Bug 2 — Ketika `pathSize % 2 == 1` (path panjang ganjil), kode mencetak `path[pathSize/2] * -1` — dikalikan -1, ini jelas salah.

Bug 3 — `farthestNode` dipanggil pertama kali dari node `0`, tapi tree dibangun dengan node `1, 2, 3` dan tidak ada node `0` terhubung ke `1` (sebenarnya ada edge `1-0`, jadi `0` ada). Ini oke, tapi perlu diwaspadai kalau struktur tree berubah.

Kode Setelah Diperbaiki

Cell 1 — Inisialisasi global:

```
# To create tree
tree = {}
path = []
maxHeight, maxHeightNode = -1, -1
```

Cell 2 — getDiameterPath (tidak ada bug):

```
def getDiameterPath(vertex, targetVertex, parent, path):
    if (vertex == targetVertex):
        path.append(vertex)
        return True

    for i in range(len(tree[vertex])):
        if (tree[vertex][i] == parent):
            continue
        if (getDiameterPath(tree[vertex][i], targetVertex, vertex, path)):
            path.append(vertex)
```

```
            return True
    return False
```

Cell 3 — farthestNode (tidak ada bug):

```
def farthestNode(vertex, parent, height):
    global maxHeight, maxHeightNode

    if (height > maxHeight):
        maxHeight = height
        maxHeightNode = vertex

    if (vertex in tree):
        for i in range(len(tree[vertex])):
            if (tree[vertex][i] == parent):
                continue
            farthestNode(tree[vertex][i], vertex, height + 1)
```

Cell 4 — addedge (tidak ada bug):

```
def addedge(a, b):
    if (a not in tree):
        tree[a] = []
    tree[a].append(b)

    if (b not in tree):
        tree[b] = []
    tree[b].append(a)
```

Cell 5 — FindCenter, ini yang utama diperbaiki:

```
def FindCenter(n):
    global maxHeight, maxHeightNode

    # Bug 1 diperbaiki: pakai global, bukan variabel lokal
    maxHeight = -1
    maxHeightNode = -1

    farthestNode(0, -1, 0)
    leaf1 = maxHeightNode

    # Reset lagi untuk pencarian kedua
    maxHeight = -1
    farthestNode(leaf1, -1, 0)

    leaf2 = maxHeightNode

    path = []
    getDiameterPath(leaf1, leaf2, -1, path)

    pathSize = len(path)

    if (pathSize % 2 == 1):
        # Bug 2 diperbaiki: hapus * -1, cukup cetak node tengahnya
        print(path[int(pathSize / 2)])
    else:
        print(path[int(pathSize / 2)], ", ", path[int((pathSize - 1) / 2)], sep="", end="")
```

Cell 6 — Main (tidak ada bug):

```
N = 4

tree = {}
addedge(1, 0)
addedge(1, 2)
addedge(1, 3)

FindCenter(N)
```

Output

Struktur tree-nya seperti ini:

```
    0
    |
    1
```

```
  / \
 2   3
```

Node `1` terhubung ke `0`, `2`, dan `3`. Diameter pohon ini misalnya dari `0` ke `2` atau `0` ke `3`, panjang path = 3 node. Titik tengahnya adalah node `1`.

```
1
```

Node `1` adalah center dari pohon tersebut karena posisinya paling sentral — semua node lain berjarak maksimal 1 langkah darinya.

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/

Nama: Muhammad Rafly Romeo Nasution
Kelas: 3KA25
NPM: 10123875

MK Praktikum Unggulan Universitas Gunadarma

Mata Kuliah: Praktikum Terapan Teori Graf (Tingkat 3)

Pertemuan VII

```
import sys
```

```
class Graph:
    # Constructor
    def __init__(self, edges, n):

        # A list of lists to represent an adjacency list
        self.adjList = [[] for _ in range(n)]

        # Add edges to the directed graph
        for (source, dest, weight) in edges:
            self.adjList[source].append((dest, weight))

# Perform DFS on the graph and set the departure time of all vertices of the graph
```

```
def DFS(graph, v, discovered, departure, time):

    # Mark the current node as discovered
    discovered[v] = True

    # Set arrival time - not needed
    # time = time + 1

    # Do for every edge (v, u)
    for (u, w) in graph.adjList[v]:
        # if `u` is not yet discovered
        if not discovered[u]:
            time = DFS(graph, u, discovered, departure, time)

    # Ready to backtrack
    # Set departure time of vertex `v`
    departure[time] = v

    time = time + 1
    return time
```

```
# The function performs the topological sort on a given DAG and then finds
# the longest distance of all vertices from the given source by running one pass
# of the Bellman-Ford algorithm on edges of vertices in topological order
def findShortestDistance(graph, source, n):

    # `departure` stores the vertex number using departure time as an index
    departure = [-1] * n

    # To keep track of whether a vertex is discovered or not
    discovered = [False] * n
    time = 0

    # Perform DFS on all undiscovered vertices
    for i in range(n):
        if not discovered[i]:
            time = DFS(graph, i, discovered, departure, time)

    cost = [sys.maxsize] * n
    cost[source] = 0

    # Process the vertices in topological order, i.e., in order of their decreasing departure time in DFS
    for i in reversed(range(n)):

        # For each vertex in topological order, relax the cost of its adjacent vertices
        v = departure[i]

        # Edge from `v` to `u` having weight `w`
        for (u, w) in graph.adjList[v]:
            # if the distance to destination `u` can be shortened by
            # taking edge (v, u), update cost to the new lower value
            if cost[v] != sys.maxsize and cost[v] + w < cost[u]:
                cost[u] = cost[v] + w

    # Print shortest paths
```

```
    for i in range(n):
        print(f'dist({source}, {i}) = {cost[i]}')
```

```
if __name__ == '__main__':

    # List of graph edges as per the above diagram
    edges = [
        (0, 6, 2), (1, 2, -4), (1, 4, 1), (1, 6, 8), (3, 0, 3), (3, 4, 5),
        (5, 1, 2), (7, 0, 6), (7, 1, -1), (7, 3, 4), (7, 5, -4)
    ]

    # Total number of nodes in the graph (labelled from 0 to 7)
    n = 8

    # Build a graph from the given edges
    graph = Graph(edges, n)

    # Source vertex
    source = 7

    # Find the shortest distance of all vertices from the given source
    findShortestDistance(graph, source, n)
```

```
dist(7, 0) = 6
dist(7, 1) = -2
dist(7, 2) = -6
dist(7, 3) = 4
dist(7, 4) = -1
dist(7, 5) = -4
dist(7, 6) = 6
dist(7, 7) = 0
```

Logika Program

Program ini mencari **jalur terpendek (shortest path) dari satu sumber ke semua node** pada graf berarah tanpa siklus (DAG — Directed Acyclic Graph). Algoritma yang dipakai adalah kombinasi **Topological Sort via DFS** lalu **satu kali relaksasi Bellman-Ford**.

Langkah 1 — Bangun Graf

Kelas `Graph` menerima daftar edge berformat `(sumber, tujuan, bobot)` dan menyimpannya sebagai adjacency list. Hasilnya seperti ini untuk beberapa node:

```
node 7 -> [(0,6), (1,-1), (3,4), (5,-4)]
node 5 -> [(1,2)]
node 1 -> [(2,-4), (4,1), (6,8)]
...
```

Langkah 2 — DFS untuk Topological Sort

Fungsi `DFS` menelusuri graf secara rekursif. Setiap node dicatat **waktu keluarnya (departure time)** — yaitu saat semua tetangganya sudah selesai diproses. Node yang selesai paling akhir berarti tidak bergantung pada siapapun, sehingga dia harus diproses pertama dalam topological order.

Array `departure` menyimpan `departure[time] = node`, sehingga kalau dibaca dari belakang (`reversed`) itulah urutan topologisnya.

Langkah 3 — Relaksasi Jarak

Setelah urutan topologis didapat, program menelusuri setiap node sesuai urutan tersebut dan melakukan **relaksasi**:

```
jika cost[v] + bobot_edge < cost[u]:
    perbarui cost[u]
```

Karena ini DAG dan urutannya topologis, setiap node pasti sudah diproses sebelum tetangganya — sehingga cukup satu kali lintasan saja, berbeda dengan Dijkstra atau Bellman-Ford penuh yang butuh iterasi berulang.

Nilai awal semua `cost` adalah tak hingga (`sys.maxsize`), kecuali node sumber yang bernilai `0`.

Struktur Graf

```
7 --(-1)--> 1 --(-4)--> 2
7 --(-4)--> 5 --( 2)--> 1
7 --( 4)--> 3 --( 3)--> 0 --( 2)--> 6
7 --( 6)--> 0
            1 --( 1)--> 4
            1 --( 8)--> 6
            3 --( 5)--> 4
```

Output

```
dist(7, 0) = 6
dist(7, 1) = -2
```

```
dist(7, 2) = -6
dist(7, 3) = 4
dist(7, 4) = -1
dist(7, 5) = -4
dist(7, 6) = 6
dist(7, 7) = 0
```

Penjelasan beberapa jalur terpendeknya:

Tujuan	Jalur	Total
0	7 → 0 (bobot 6)	6
1	7 → 5 → 1 (-4 + 2)	-2
2	7 → 5 → 1 → 2 (-4 + 2 + -4)	-6
5	7 → 5 (-4)	-4
7	sumber itu sendiri	0

Node `2` punya jarak paling negatif karena jalur lewat node `5` dan `1` mengakumulasi bobot negatif, yang memang diperbolehkan di DAG (berbeda dengan Dijkstra yang tidak bisa menangani bobot negatif).

Copyright © Pengelola MK Praktikum Unggulan (Praktikum DGX), Universitas Gunadarma

https://www.praktikum-hpc.gunadarma.ac.id/